

Department of Computer Applications
II MCA – Object Oriented Programming using Java
Static Members + Encapsulation + Abstraction
Case-Study / Scenario Based MCQs

25MX308
Dayananda J

Q1 – Bank System (Encapsulation + Static counter)

A bank wants to:

- hide account balance
- auto-generate account numbers
- track total accounts created

```
class Account {  
    private double balance;  
    static int totalAccounts = 0;
```

```
    Account(double b) {  
        balance = b;  
        totalAccounts++;  
    }  
}
```

Which improvement best follows **encapsulation principle**?

- A. Make balance public
- B. Make totalAccounts public
- C. Provide getter/setter for balance**
- D. Remove static

Q2 – University Portal (Static shared data)

All students share one **university name**, but each has different roll number.

Best design?

- A. String universityName;
- B. static String universityName;**
- C. final int roll;
- D. private static roll;

Q3 – Library System (Execution order)

```
class Library {  
    static {  
        System.out.print("Load ");  
    }  
  
    Library() {  
        System.out.print("Create ");  
    }  
}
```

If 3 objects are created, output?

- A. Load Create Create Create**
- B. Create Create Create
- C. Load Load Create
- D. Create Load Create

Q4 – Payment App (Abstraction)

You design:

```
abstract class Payment {  
    abstract void pay(double amount);  
}
```

Why abstract?

- A. To store static data
 - B. To prevent object creation**
 - C. To improve speed
 - D. To allow multiple constructors
-

Q5 – Employee Salary Protection (Encapsulation bug)

```
class Employee {  
    public double salary;  
}
```

Problem with this design?

- A. Slower execution
 - B. Breaks data hiding**
 - C. Cannot create objects
 - D. Uses extra memory
-

Q6 – Logger Utility (Static methods)

A logging class is used across the application:

```
Logger.log("Error");
```

Why should log() be static?

- A. Needs object state
 - B. Shared utility behavior**
 - C. Uses constructor
 - D. Uses inheritance
-

Q7 – Factory Counter (Static tracing)

```
class Product {  
    static int count = 0;
```

```
    Product() {  
        count++;  
    }  
}
```

After:

```
Product p1 = new Product();
```

```
Product p2 = new Product();
```

```
Product p3 = new Product();
```

What does count represent?

- A. Memory size
 - B. Number of classes
 - C. Total products created**
 - D. Instance variable copies
-

Q8 – Secure Database Config (Static block)

You want to load configuration **once** when application starts.

Best approach?

- A. Constructor
 - B. Instance method
 - C. Static block**
 - D. Getter method
-

Q9 – Package Access (Access specifier reasoning)

A field must be:

- accessible inside package
- hidden outside package

- not necessarily subclass specific

Best modifier?

- A. private
- B. public
- C. protected
- D. default

Q10 – Vehicle Design (Abstraction + Encapsulation)

```
abstract class Vehicle {  
    private int speed;  
  
    public void setSpeed(int s) {  
        if(s >= 0)  
            speed = s;  
    }  
  
    abstract void move();  
}
```

Which concepts are applied?

- A. Only inheritance
- B. Only polymorphism
- C. Encapsulation + Abstraction
- D. Static + Constructor

Q11 – Employee Hierarchy (Inheritance)

A company has different employee types but all employees share common attributes like **name and employeeid**.

```
class Employee {  
    String name;  
    int empld;  
}  
  
class Manager extends Employee {  
    int teamSize;  
}
```

What concept is being applied?

- A. Encapsulation
- B. Inheritance
- C. Composition
- D. Interface

Q12 – Method Overriding (Runtime Polymorphism)

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes sound");  
    }  
}
```

```
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

When executed:

```
Animal a = new Dog();
```

```
a.sound();
```

What is the output?

- A. Animal makes sound
 - B. Dog barks
 - C. Compilation error
 - D. Runtime error
-

Q13 – Aggregation Example (University System)

A **Department** has many **Students**, but students can exist even if the department object is destroyed.

```
class Student {  
    String name;  
}
```

```
class Department {  
    Student s;  
}
```

This relationship is best described as:

- A. Composition
 - B. Aggregation
 - C. Inheritance
 - D. Interface
-

Q14 – Composition Example (Car System)

A **Car** cannot exist without an **Engine**.

```
class Engine {  
}
```

```
class Car {  
    private Engine engine;  
  
    Car() {  
        engine = new Engine();  
    }  
}
```

This design represents:

- A. Aggregation
 - B. Composition
 - C. Inheritance
 - D. Encapsulation
-

Q15 – Interface Design (Payment Gateway)

Multiple payment methods must implement a **common method pay()**.

```
interface Payment {  
    void pay(double amount);  
}
```

Why is an **interface** appropriate here?

- A. To store object data
 - B. To enforce common behavior across classes
 - C. To reduce memory usage
 - D. To create static objects
-

Q16 – Interface Implementation

```
interface Printer {  
    void print();  
}
```

```
class LaserPrinter implements Printer {  
    public void print() {  
        System.out.println("Laser printing");  
    }  
}
```

```
}  
}
```

Which concept is demonstrated?

- A. **Interface implementation**
 - B. Constructor chaining
 - C. Aggregation
 - D. Static binding
-

Q17 – Overriding Rule

```
class Shape {  
    void draw() {  
        System.out.println("Drawing shape");  
    }  
}
```

```
class Circle extends Shape {  
    void draw() {  
        System.out.println("Drawing circle");  
    }  
}
```

This is an example of:

- A. Method overloading
 - B. **Method overriding**
 - C. Constructor overriding
 - D. Static method
-

Q18 – Abstract Class Usage (Vehicle System)

```
abstract class Vehicle {  
    abstract void start();  
}
```

Why declare Vehicle as abstract?

- A. To store static variables
 - B. **To force subclasses to implement start()**
 - C. To increase execution speed
 - D. To avoid inheritance
-

Q19 – Multiple Behavior Requirement

A **SmartPhone** must support **Camera** and **MusicPlayer** behaviors.

Best design?

- A. Use two abstract classes
 - B. Use multiple inheritance of classes
 - C. **Use multiple interfaces**
 - D. Use composition only
-

Q20 – Aggregation vs Composition Reasoning

Consider:

```
class Book {  
}
```

```
class Library {  
    Book b;  
}
```

If a **Book** can exist independently of **Library**, the relationship is:

- A. Composition
- B. **Aggregation**
- C. Inheritance
- D. Interface

